Design a Scalable Agentic System

Submission report - Datazoic AI Engineer task

Scope. A working platform that lets a user drive 50+ PayPal APIs by chat, plus a RAG tool and a system-introspection tool, designed so that the number of registered tools is not a constraint on accuracy. Architecture, workflow, technical reference and user manual accompany this report.

1. The problem

Tool-calling accuracy degrades as the tool count grows: the model picks semantic neighbours, borrows parameters across schemas, and pays for a context window full of irrelevant definitions. The constraint is the size of the choice set, so the design removes catalog size from the model's view of a turn.

2. Agent structure

A single agent as an explicit LangGraph state machine of ten named stages, rather than a crew of role-playing agents or a free-form ReAct loop. The problem is tool selection within one agent, which multi-agent abstractions do not address; and a financial action needs a deterministic point at which approval is enforced, which a free-form loop makes hard to guarantee. The cost is more explicit wiring.

3. Tool selection and routing

Retrieval narrows the catalog before the model is involved: domain classification, then dense vector search alongside sparse keyword matching, fused with Reciprocal Rank Fusion, then prerequisite expansion. The resulting shortlist of roughly ten tools is translated into function declarations carrying each parameter's type and required flag. The model picks one tool and fills its arguments; a name outside the shortlist is rejected in code, and a required argument that cannot be grounded produces a question to the user rather than an invented value.

> **KEY POINT**
> This is the core of the answer. The catalog can grow without bound because what reaches the model is bounded by top-k, not by the number of registered tools.

4. State management

A typed state object carries identifiers, intent, entities, the candidate shortlist, the plan, approval status, the execution trace, token accounting and the timeline between stages. Two concerns are separated deliberately: LangGraph checkpoints, scoped per run, exist so a run suspended for human approval can be resumed - durably, surviving a restart; conversation history lives in the relational store and is replayed into each new turn, so it is independent of checkpoint lifetime and visible to the reporting screens. Scoping checkpoints per run also means a suspended run cannot be corrupted by the next message in the same conversation.

5. Scalability

Concern	How it scales
Prompt size	Flat in catalog size; bounded by top-k
Adding a service	Ingest, register, index. No prompt or code change
Collisions across services	Hierarchical namespaces, scoped per intent
Governance of new endpoints	Risk tier and approval policy are per-tool columns, so a new mutating endpoint inherits the gate
Retrieval latency	Indexed vector search with a domain payload filter

6. Error handling

- **Before dispatch:** required arguments are validated against the tool contract; a missing mandatory field stops the turn with a question instead of a guess.
- **At dispatch:** mutating calls carry a stable idempotency identifier, so a retry after a timeout cannot double-charge.
- **After a failure:** the error payload returns to the planner, which can retry with corrected arguments or consult the RAG tool about the error code.
- **Dependency failure:** every external dependency has a fallback, so the platform runs with no key, no network and no external services.
- **Bounded loops:** an iteration budget caps tool calls per turn.
- **Audit:** failures are recorded with the same fidelity as successes.

7. Observability

Each turn emits a stage-by-stage timeline, surfaced in the chat screen, alongside relational audit tables: one row per run, one per tool attempt, one per token record, one per approval decision. Joining them on the per-turn request identifier is what lets a task be reported with its own token cost and its own tool calls.

LangSmith is wired in and optional, as the task suggested. With a key configured, LangGraph contributes a span per pipeline node and the two reasoning calls are traced explicitly, carrying the prompt, the candidate shortlist, the selected tool and the token counts. With no key, tracing is off and the decorator is a no-op, so no part of the call path depends on it being installed or reachable.

8. What runs today

Capability	Status
Chat-driven orchestration over the catalog	Working
Two-stage hybrid retrieval with RRF and dependency expansion	Working
Gemini function calling over the bounded shortlist	Working; not covered by automated tests
Grounded answer synthesis from the execution trace	Working; not covered by automated tests
Model-driven multi-step replanning	Working; not covered by automated tests
Deterministic fallback with no API key	Working and tested
Human approval with durable suspend and resume	Working and tested
RAG tool over an extensible knowledge base	Working; Gemini embeddings when a key is set, local fallback otherwise
System Search over catalog and logs	Working and tested
Role-based access control and per-user isolation	Working and tested
Token accounting from SDK-reported counts	Working
MCP integration	Working
LangSmith tracing (optional, key-gated)	Working and tested
Knowledge-base deletion removes its vectors	Working and tested

The automated suite runs offline and covers the pipeline, retrieval, governance, approval lifecycle, RBAC and the planner's contract against a stubbed SDK. The live model path is exercised manually by configuring a key; it is called out as untested rather than claimed as verified.

9. What I would do next

- **Evaluate retrieval directly.** A labelled set of query-to-tool pairs, measuring whether the correct tool appears in the shortlist, separates retrieval quality from model quality - the number that matters most as the catalog grows.
- **Re-rank the shortlist** with a cross-encoder when a domain holds many near-identical endpoints.
- **Add a reindex command**, so switching embedding provider does not require deleting and re-ingesting each source by hand.
- **Broaden the fallback router**, or drop it once a key is always available, so document questions route on meaning rather than trigger words.
- **Replace ad-hoc SQLite column migration with a migration tool** before any deployment that must preserve data.